

Automated Defect Detection System

Product Case Study

Designing UX Where Software Controls Physical Motion

Introduction

I worked on an Automated Defect Detection system used in a **manufacturing environment**.

The system combines:

- Physical image capture (robot / camera)
- AI-based defect inference
- Human inspection and confirmation

Unlike typical software products, this system had hardware dependencies, AI maturity constraints, and safety-critical transitions, all of which had direct UX implications.

Problem Statement

As automated inspection systems evolve, users must make confident decisions while interacting with incomplete, asynchronous, and safety-critical system states.

The challenge was to design a review experience that clearly communicates what the system is doing across image capture, AI inference, and physical motion—while supporting phased deployment and future automation.

BUT this was not a “screen design” problem!

Early on, it became clear that the main challenge was not layout or interaction patterns, but:

“Making complex, asynchronous system behavior understandable and trustworthy for users”

The system could be in multiple valid states at the same time:

- Image captured, but AI not available
- Image captured, AI running
- Image captured, AI finished
- Robot idle vs robot in motion

If these states were not explicitly communicated, users would lose trust—or worse, face safety risks.

My Design Approach

Instead of designing a single end-state experience, I intentionally decoupled the UX for image capture and inference detection, treating them as independent but connected modules.

This allowed the system to:

- Ship value early
- Remain honest about its capabilities
- Scale without redesigning core flows

1. Designing for Incremental Delivery

From product and engineering discussions, we understood –

- Image capture would ship first
- Inference detection would be deployed in a later sprint
- AI availability could vary based on environment, model readiness, or rollout phase

This meant the UX could not assume AI always existed.

2. State-Driven UX Instead of Feature-Driven UX

Rather than hiding incomplete functionality, we designed the UI to reflect the system’s actual state.

Explicit system states included:

- Image captured
- Image processing
- Inference pending (image ready, AI not started)
- Inference processing
- Inference ready

Each state had:

- Clear labels
- Distinct visual treatment
- Defined user actions

This removed ambiguity and reduced user anxiety during partial rollouts.

3. Human-in-the-Loop Without Blocking the Future

During early phases:

- Inspectors needed to edit, add, or delete defects
- AI outputs needed validation

But the long-term vision was automation with confidence.

Rather than baking assumptions into the UI, we defined feature availability based on system maturity, ensuring:

- Manual workflows remained usable without AI
- AI-assisted workflows could be layered in later
- Editing could be disabled once the model matured

This reduced long-term UX debt and avoided rework.

Outcome

The final experience:

- Supported phased rollout without redesign
- Reduced cognitive load during inspection
- Made safety-critical transitions explicit
- Scaled cleanly from manual → AI-assisted → automated inspection

My Learnings

This project reinforced that in complex, AI-driven systems:

Good UX is about making system behavior legible, not just making screens aesthetic.

Designing for incremental delivery turned out to be a UX strength not a constraint.